

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 8

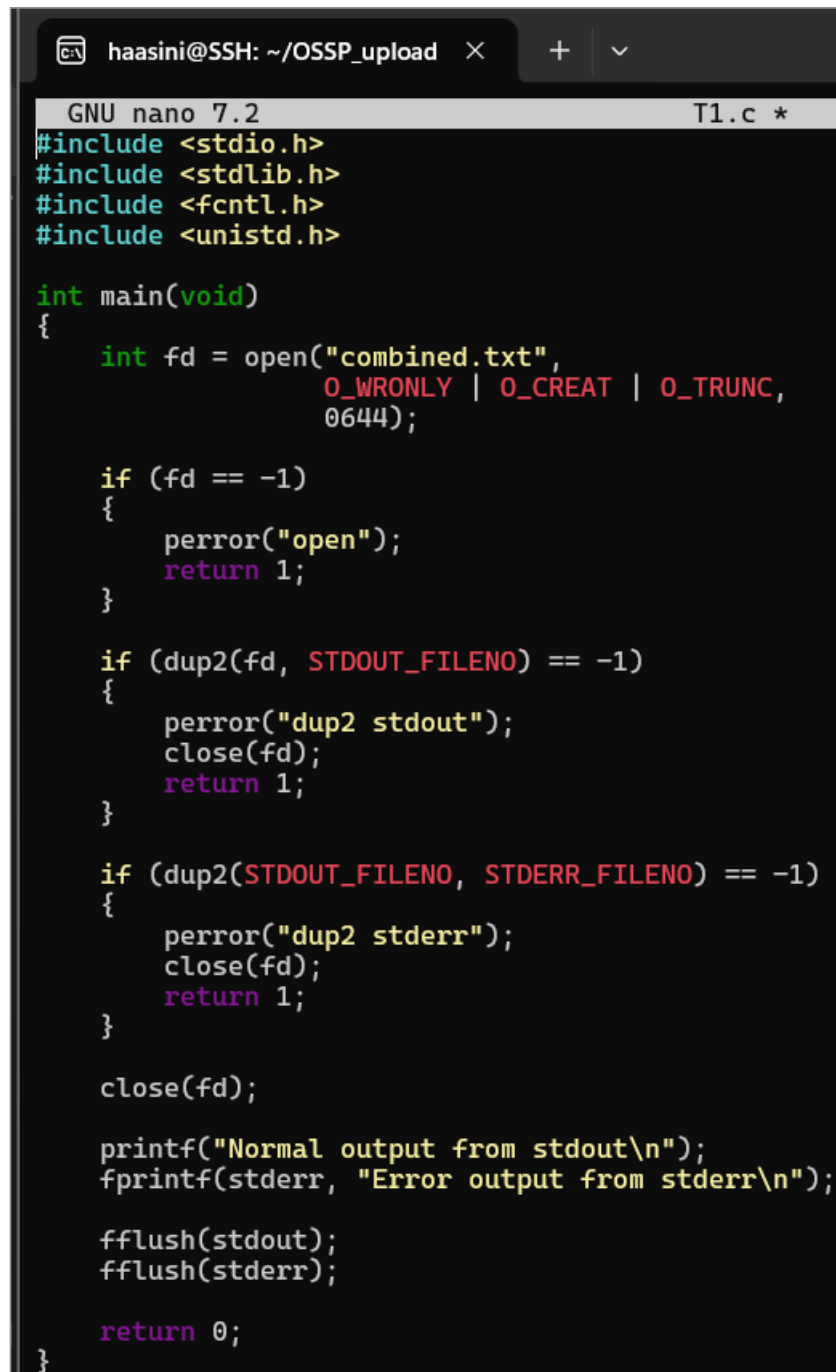
Name: S Svara Haasini Roll No: 2520090170

Task 1: Combined stdout and stderr Redirection

Question:

Parse combined redirection, duplicate file descriptors, merge standard output and standard error streams, verify stream behavior, handle edge cases, and test integrated output.

Source Code:



```
GNU nano 7.2 T1.c *
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(void)
{
    int fd = open("combined.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2 stdout");
        close(fd);
        return 1;
    }

    if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1)
    {
        perror("dup2 stderr");
        close(fd);
        return 1;
    }

    close(fd);

    printf("Normal output from stdout\n");
    fprintf(stderr, "Error output from stderr\n");

    fflush(stdout);
    fflush(stderr);

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week8$ nano T1.c
haasini@SSH:~/OSSP_upload/Skill_Week8$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week8$ ./T1
cat combined.txt
Error output from stderr
Normal output from stdout
```

Observations:

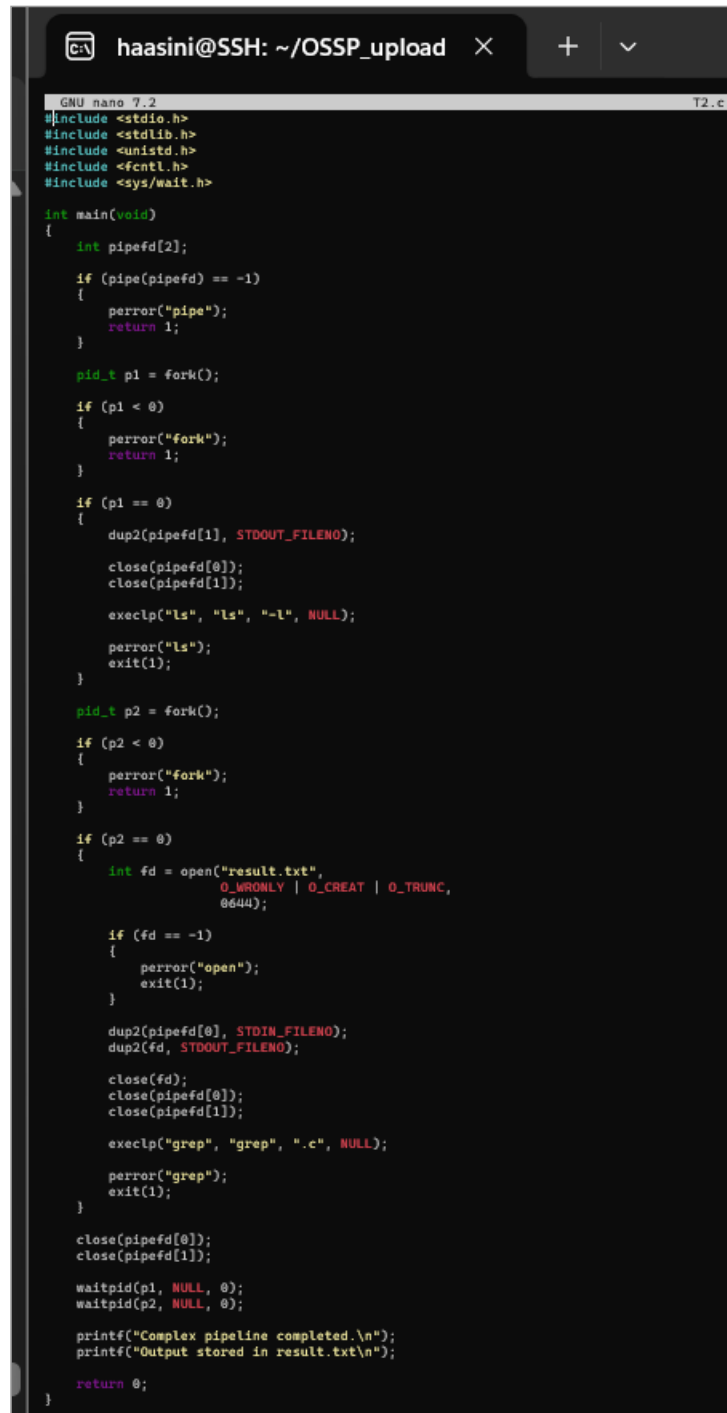
1. `open()` opens `combined.txt` with `O_WRONLY` | `O_CREAT` | `O_TRUNC` and mode `0644`: write-only, created if it does not exist, and truncated if it already exists. A return value of `-1` is reported with `perror()`.
2. `dup2(fd, STDOUT_FILENO)` makes file descriptor 1 (standard output) refer to the opened `combined.txt`. `dup2()` only duplicates/reassigns a descriptor; the file itself is created by `open()`.
3. `dup2(STDOUT_FILENO, STDERR_FILENO)` makes file descriptor 2 (standard error) refer to the same destination as descriptor 1. Both streams are therefore intentionally merged into one destination, `combined.txt`, not into separate files.
4. Each `dup2()` call is checked for `-1`; on failure the program prints an error with `perror()`, closes `fd` and returns `1`. After both duplications, `close(fd)` releases the original descriptor, while descriptors 1 and 2 still refer to `combined.txt`.
5. `printf("Normal output from stdout\n")` writes to `stdout`, and `fprintf(stderr, "Error output from stderr\n")` writes to `stderr`. `fflush(stdout)` and `fflush(stderr)` then flush both streams.
6. The program was compiled with `gcc -Wall -Wextra T1.c -o T1` and run as `./T1`; no program output appears on the terminal after `./T1`, because both streams were redirected to the file.
7. `cat combined.txt` shows both messages, "Error output from stderr" and "Normal output from stdout", confirming that normal output and error output were both stored in `combined.txt`.
8. In the file the `stderr` line appears before the `stdout` line even though `printf()` is called first. This is consistent with `stdout` being buffered when redirected to a file (written when `fflush(stdout)` runs) while `stderr` is unbuffered (written immediately).

Task 2: Complex Pipeline with Output Redirection

Question:

Parse a complex command, combine pipes and redirections, resolve execution order, connect processes, validate the execution plan, and test a pipeline whose final output is redirected to a file.

Source Code:



```
haasini@SSH: ~/OSSP_upload × + v
GNU nano 7.2 T2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main(void)
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("ls");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (p2 == 0)
    {
        int fd = open("result.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

        if (fd == -1)
        {
            perror("open");
            exit(1);
        }

        dup2(pipefd[0], STDIN_FILENO);
        dup2(fd, STDOUT_FILENO);

        close(fd);
        close(pipefd[0]);
        close(pipefd[1]);

        execlp("grep", "grep", ".c", NULL);

        perror("grep");
        exit(1);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);

    printf("Complex pipeline completed.\n");
    printf("Output stored in result.txt\n");

    return 0;
}
```

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week8$ nano T2.c
haasini@SSH:~/OSSP_upload/Skill_Week8$ gcc -Wall -Wextra T2.c -o T2
./T2
cat result.txt
Complex pipeline completed.
Output stored in result.txt
-rw-r--r-- 1 haasini haasini 696 Sep 18 12:34 T1.c
-rw-r--r-- 1 haasini haasini 1291 Sep 18 12:36 T2.c
-rw-r--r-- 1 haasini haasini 51 Sep 18 12:34 combined.txt
haasini@SSH:~/OSSP_upload/Skill_Week8$ |

```

Pipeline flow (equivalent of `ls -l | grep ".c" > result.txt`):

```

ls -l
|
v
PIPE
|
v
grep ".c"
|
v
result.txt

```

Observations:

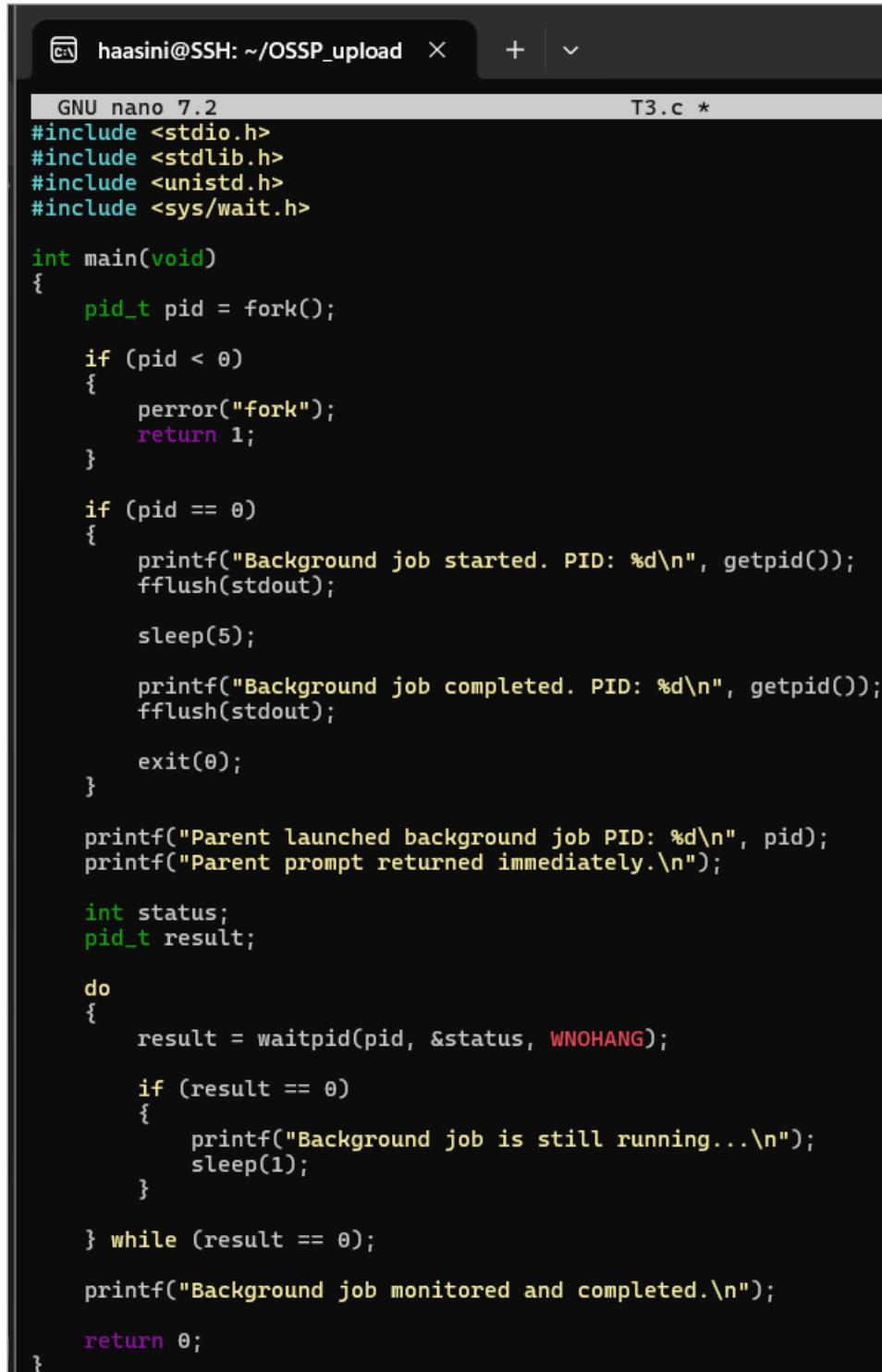
1. `pipe(pipefd)` creates a kernel communication channel: `pipefd[0]` is the read end and `pipefd[1]` is the write end. `pipe()` only creates the channel; it does not execute any command.
2. `fork()` is called twice, creating two child processes (`p1` and `p2`). Each call is checked for failure.
3. The first child (`p1 == 0`) uses `dup2(pipefd[1], STDOUT_FILENO)` to redirect its stdout into the pipe, closes both pipe ends, and then `execvp("ls", "ls", "-l", NULL)` replaces the child with `ls -l`.
4. The second child (`p2 == 0`) opens `result.txt` for output with `O_WRONLY | O_CREAT | O_TRUNC` and mode `0644`.
5. In the second child, `dup2(pipefd[0], STDIN_FILENO)` makes stdin read from the pipe, and `dup2(fd, STDOUT_FILENO)` redirects its stdout to `result.txt`.
6. The second child then closes `fd`, `pipefd[0]` and `pipefd[1]`, and `execvp("grep", "grep", ".c", NULL)` replaces it with `grep ".c"`. Unused descriptors are also closed in the parent.
7. The parent uses `waitpid(p1, NULL, 0)` and `waitpid(p2, NULL, 0)` to wait for both children, then prints "Complex pipeline completed." and "Output stored in result.txt".
8. `cat result.txt` verifies the redirected pipeline output: the lines for `T1.c`, `T2.c` and `combined.txt` appear. In a regular expression the `.` in `".c"` matches any character, so `.c` also matches the space-then-`"c"` in the `combined.txt` line.
9. Overall the program implements the equivalent of `ls -l | grep ".c" > result.txt`, following the flow shown above.

Task 3: Background Process and Non-Blocking Monitoring

Question:

Detect and launch a background job, allow the parent process to continue without blocking, store job information, monitor execution, and verify completion of the background task.

Source Code:



```
GNU nano 7.2 T3.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Background job started. PID: %d\n", getpid());
        fflush(stdout);

        sleep(5);

        printf("Background job completed. PID: %d\n", getpid());
        fflush(stdout);

        exit(0);
    }

    printf("Parent launched background job PID: %d\n", pid);
    printf("Parent prompt returned immediately.\n");

    int status;
    pid_t result;

    do
    {
        result = waitpid(pid, &status, WNOHANG);

        if (result == 0)
        {
            printf("Background job is still running...\n");
            sleep(1);
        }

    } while (result == 0);

    printf("Background job monitored and completed.\n");

    return 0;
}
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week8$ nano T3.c
haasini@SSH:~/OSSP_upload/Skill_Week8$ gcc -Wall -Wextra T3.c -o T3
./T3
Parent launched background job PID: 700
Parent prompt returned immediately.
Background job is still running...
Background job started. PID: 700
Background job is still running...
Background job is still running...
Background job is still running...
Background job is still running...
Background job is still running...
Background job completed. PID: 700
Background job monitored and completed.
haasini@SSH:~/OSSP_upload/Skill_Week8$ |
```

Observations:

1. `fork()` creates the child process, which represents the background job. A negative return is handled with `perror("fork")`; a return of 0 identifies the child.
2. The child prints "Background job started. PID: 700", flushes stdout, simulates work with `sleep(5)`, then prints "Background job completed. PID: 700" and calls `exit(0)`. The child runs independently of the parent.
3. The parent receives the child's PID from `fork()` and prints "Parent launched background job PID: 700", which matches the PID printed by the child itself.
4. The parent prints "Parent prompt returned immediately." without first performing a blocking wait, so it continues executing while the child is still running.
5. The parent then loops on `waitpid(pid, &status, WNOHANG)`. `WNOHANG` makes `waitpid()` return immediately with 0 while the child has not terminated; it does not create the process (`fork()` does).
6. Whenever the result is 0, the parent prints "Background job is still running..." and sleeps 1 second before checking again. Five such messages appear in the output, interleaved with the child's own messages.
7. When the child terminates, `waitpid()` returns a non-zero value (the child's PID), the loop ends, and the parent prints "Background job monitored and completed." The parent stays alive throughout, monitoring the child with non-blocking `waitpid()` calls.

Task 4: Background Job Table and State Tracking

Question:

Create a job table, store background-process information, track job states, update job status, monitor completed jobs, and validate the final job records.

Source Code:

```

haasini@SSH: ~/OSSP_upload
GNU nano 2.9.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 3

typedef struct
{
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main(void)
{
    Job jobs[MAX_JOBS];

    printf("=== Background Job Table ===\n");

    for (int i = 0; i < MAX_JOBS; i++)
    {
        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            return 1;
        }

        if (pid == 0)
        {
            sleep(1 + 2);
            exit(0);
        }

        jobs[i].job_id = i + 1;
        jobs[i].pid = pid;

        sprintf(jobs[i].state,
            sizeof(jobs[i].state),
            "Running");
    }

    printf("\nInitial Job Table:\n");
    printf("JOB\tPID\tSTATE\n");

    for (int i = 0; i < MAX_JOBS; i++)
    {
        printf("%d\t%d\t%s\n",
            jobs[i].job_id,
            jobs[i].pid,
            jobs[i].state);
    }

    int remaining = MAX_JOBS;

    while (remaining > 0)
    {
        for (int i = 0; i < MAX_JOBS; i++)
        {
            if (jobs[i].pid == 0)
                continue;

            pid_t result = waitpid(jobs[i].pid, NULL, WNOHANG);

            if (result > 0)
            {
                printf("\nJob %d (PID %d) completed.\n",
                    jobs[i].job_id,
                    jobs[i].pid);

                jobs[i].pid = 0;

                sprintf(jobs[i].state,
                    sizeof(jobs[i].state),
                    "Completed");

                remaining--;
            }
        }

        if (remaining > 0)
            sleep(1);
    }

    printf("\nFinal Job Table:\n");
    printf("JOB\tSTATE\n");

    for (int i = 0; i < MAX_JOBS; i++)
    {
        printf("%d\t%s\n",
            jobs[i].job_id,
            jobs[i].state);
    }

    printf("\nAll completed jobs validated.\n");

    return 0;
}

```

Output:

```

Background job monitored and completed.
haasini@SSH:~/OSSP_upload/Skill_Week8$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week8$ gcc -Wall -Wextra T4.c -o T4
./T4
=== Background Job Table ===

Initial Job Table:
JOB    PID    STATE
1      712    Running
2      713    Running
3      714    Running

Job 1 (PID 712) completed.

Job 2 (PID 713) completed.

Job 3 (PID 714) completed.

Final Job Table:
JOB    STATE
1      Completed
2      Completed
3      Completed

All completed jobs validated.
haasini@SSH:~/OSSP_upload/Skill_Week8$ |

```

Observations:

1. A Job structure stores `job_id`, `pid` and `state` (a 20-character string). The job table is the array `jobs[MAX_JOBS]` with `MAX_JOBS` defined as 3.
2. A loop calls `fork()` three times, creating three background children. Each child sleeps for `i + 2` seconds (2, 3 and 4 seconds) and exits with `exit(0)`.
3. For each child the parent stores `job_id = i + 1`, the child's PID, and the state "Running" (using `snprintf()`).
4. The Initial Job Table shows Job 1 with PID 712, Job 2 with PID 713 and Job 3 with PID 714, all in the Running state.
5. A `while (remaining > 0)` loop checks each job that still has a PID using `waitpid(jobs[i].pid, NULL, WNOHANG)`. It returns immediately, so a job that has not finished does not block the check of the others.
6. When `waitpid()` returns a positive value, the program prints "Job 1 (PID 712) completed.", "Job 2 (PID 713) completed." and "Job 3 (PID 714) completed." in turn, sets each state to "Completed", sets its `pid` to 0 so it is skipped in later passes, and decrements `remaining`.
7. While jobs remain, the parent calls `sleep(1)` between passes over the table.
8. The Final Job Table lists Jobs 1, 2 and 3, all in the Completed state, and the program prints "All completed jobs validated." This shows the job table tracked all three background jobs from Running to Completed.